Solution: Problem Challenge 1

Problem Statement

You are given a 2D matrix containing only 1s (land) and 0s (water).

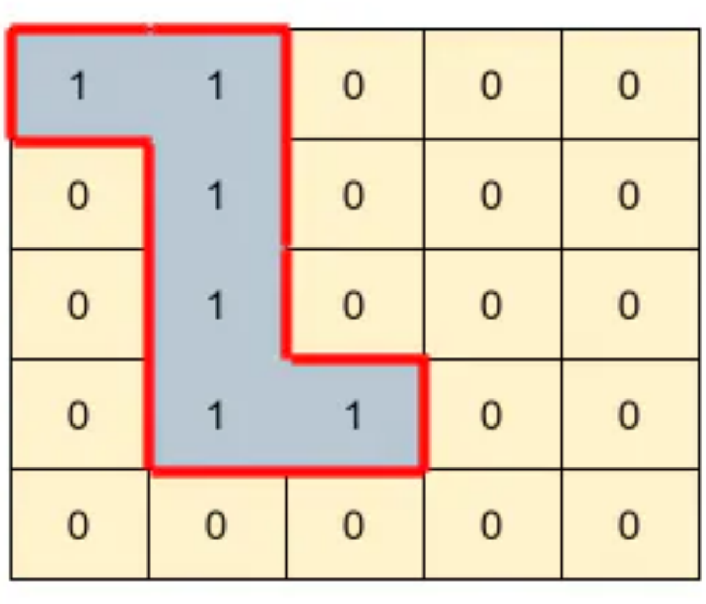
An **island** is a connected set of 1s (land) and is surrounded by either an edge or 0s (water). Each cell is considered connected to other cells horizontally or vertically (not diagonally).

There are **no lakes** on the island, so the water inside the island is not connected to the water around it. A cell is a square with a side length of 1..

The given matrix has only one island, write a function to find the perimeter of that island.

Example 1

Input: matrix =



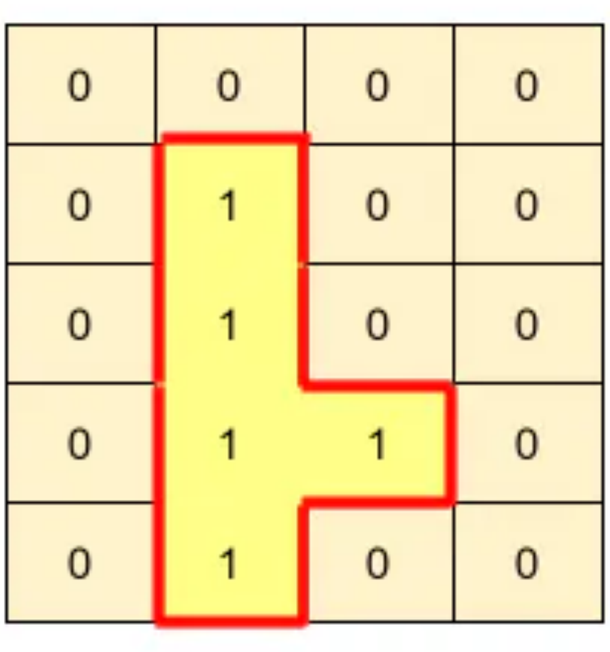
1	1	0	0	0
0	1	0	0	0
0	1	0	0	0
0	1	1	0	0
0	0	0	0	0

Output: 14

Explanation: The boundary of the island constitutes 14 sides.

Example 2

Input: matrix =



0	0	0	0
0	1	0	0
0	1	0	0
0	1	1	0
0	1	0	0

Output: 12

Explanation: The boundary of the island constitute 12 sides.

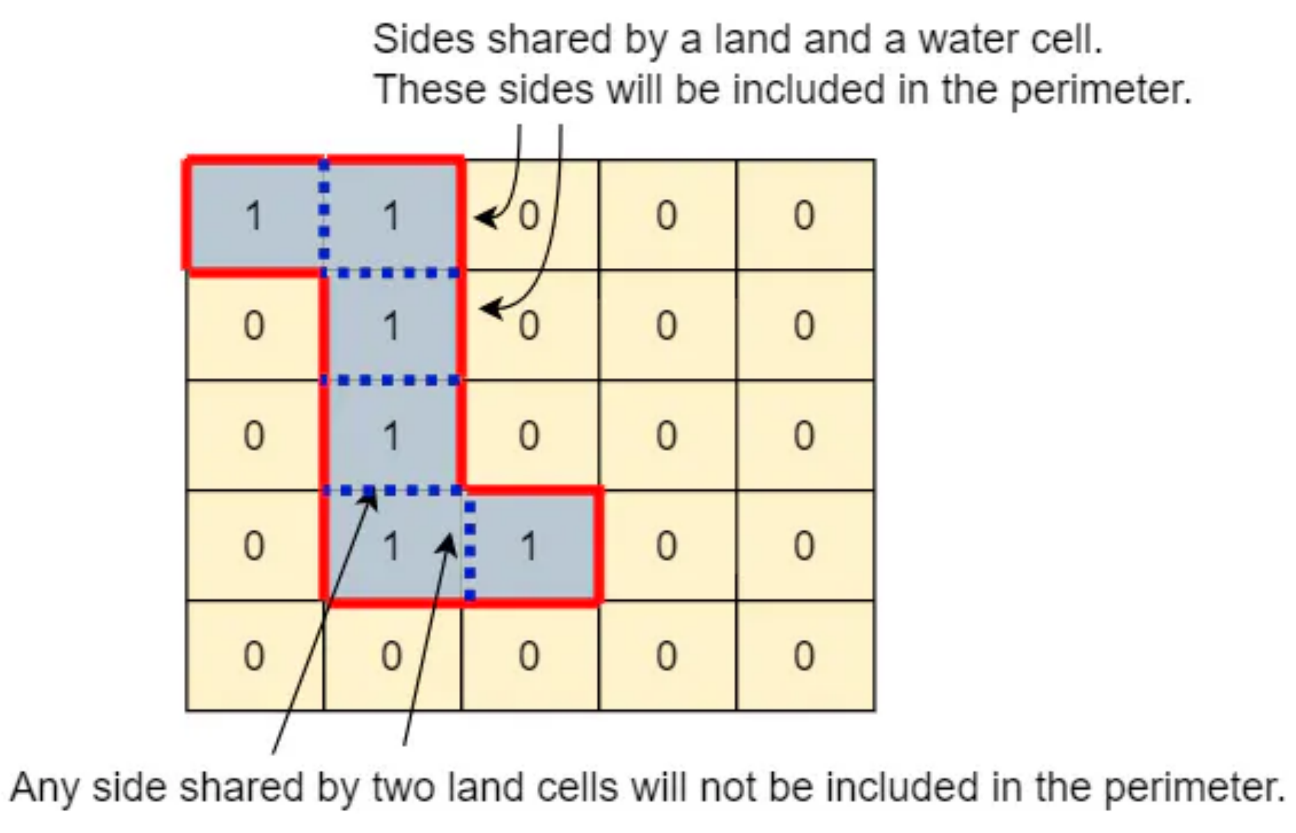
Solution

The question follows the Island pattern and is quite similar to Number of Islands problem.

We will traverse the matrix linearly to find the island. We can use the **Depth First Search (DFS)** or **Breadth First Search (BFS)** to traverse the island i.e., to find all of its connected land cells.

How do we calculate the boundary if the island?

Each cell has four sides. Each side of an island cell can be shared with another cell; we can include the side in the island perimeter only if the other cell is a water.



Sides shared by a land and a water cell.
These sides will be included in the perimeter.
Any side shared by two land cells will not be included in the perimeter.

1	1	0	0	0
0	1	0	0	0
0	1	0	0	0
0	1	1	0	0
0	0	0	0	0

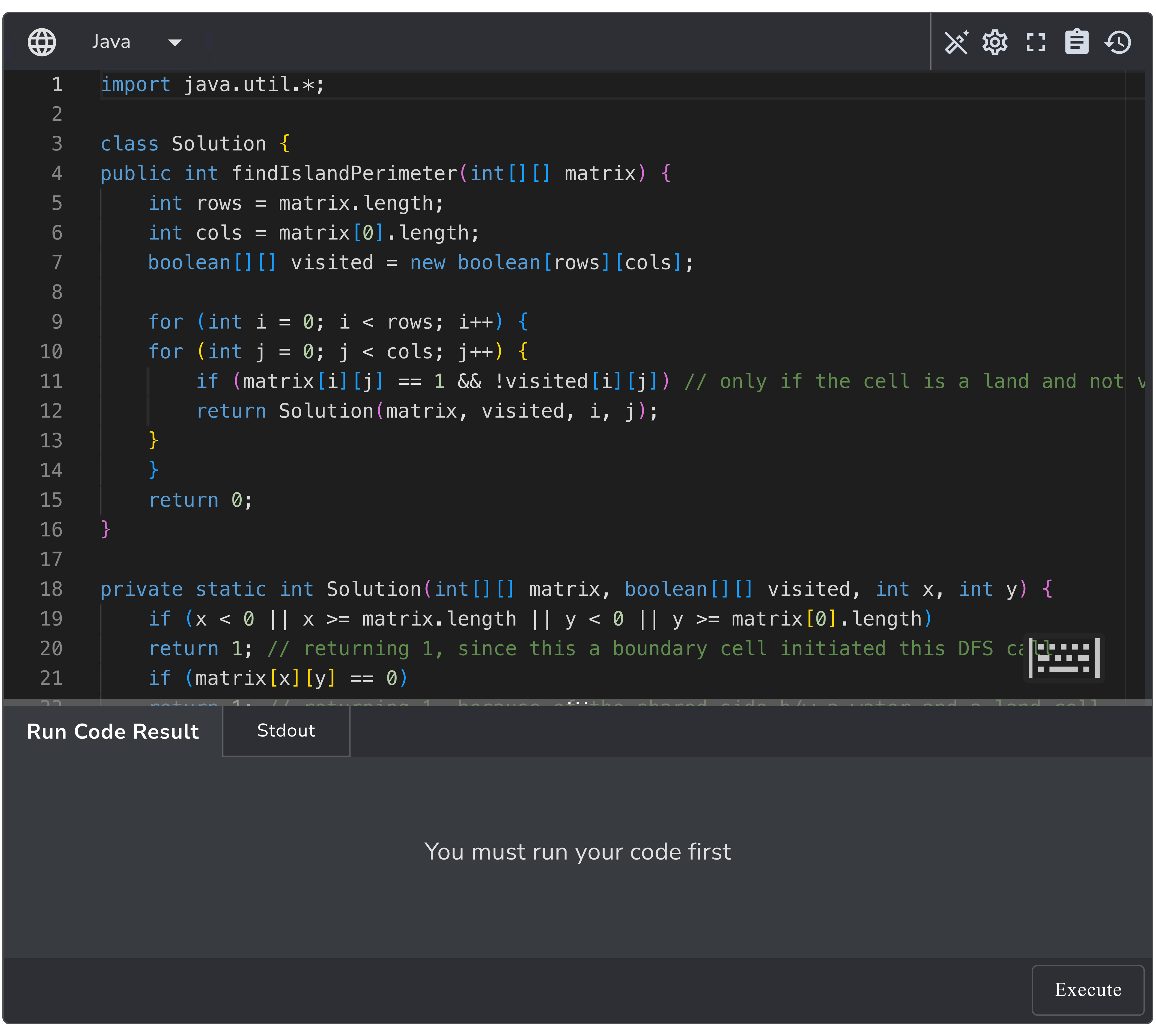
If a cell side is on boundary, we should include that side in the perimeter.

Following piece of code will cover these two conditions:

```
if (x < 0 || x >= matrix.length || y < 0 || y >= matrix[0].length)
   return 1; // returning 1, since this a boundary cell initiated this DFS call
 if (matrix[x][y] == 0)
   return 1; // returning 1, because of the shared side b/w a water and a land cell
```

Code (DFS)

Here is what our DFS algorithm will look like. We will use a separate matrix to mark nodes visited.



```
import java.util.*;

class Solution {
public int findIslandPerimeter(int[][] matrix) {
    int rows = matrix.length;
    int cols = matrix[0].length;
    boolean[][] visited = new boolean[rows][cols];

    for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        if (matrix[i][j] == 1 && !visited[i][j]) // only if the cell is a land and not
        return Solution(matrix, visited, i, j);
    }
    }
    return 0;
}

private static int Solution(int[][] matrix, boolean[][] visited, int x, int y) {
    if (x < 0 || x >= matrix.length || y < 0 || y >= matrix[0].length)
    return 1; // returning 1, since this a boundary cell initiated this DFS ca
    if (matrix[x][y] == 0)
```

You must run your code first

Time Complexity

Time complexity of the above algorithm will be $O(M * N)$, where 'M' is the number of rows and 'N' is the number of columns of the input matrix. This is due to the fact that we have to traverse the whole matrix to find the island.

Space Complexity

DFS recursion stack can go $M * N$ deep when the whole matrix is filled with '1's. Hence, the space complexity will be $O(M * N)$, where 'M' is the number of rows and 'N' is the number of columns of the input matrix.

← Previous
Problem Challenge 1 (Easy)

Next →
Problem Challenge 2 (Medium)

☐ Mark as Completed